

PortoFlow 4.0

Sistema Inteligente de Atracação — Porto do Itaqui

Análise completa do projeto · Teoria das Filas & Processos Estocásticos

Etapa 1 — Entendimento geral

Problema (desafio): o Porto do Itaqui sofre com filas de atracação por alta demanda. O sequenciamento atual não lida com incertezas (chegada aleatória, clima, tempos de operação distintos por carga, prioridades legais/contratuais, restrição de berços). A autoridade quer reduzir espera, evitar congestionamento e ociosidade, garantir atendimento prioritário e prever cenários críticos.

Solução (PortoFlow 4.0): painel de inteligência operacional que (a) **simula a fila de atracação ao vivo** (M/G/4), (b) prioriza navios por um **Índice Dinâmico de Atracação (IDA)**, (c) **compara políticas** de sequenciamento por **simulação de eventos discretos** e (d) emite **recomendações** operacionais.

Tela	Função
/ Dashboard	KPIs ao vivo (espera, fila, utilização, atraso crítico, risco climático, congestionamento)
/bercos Berços	4 servidores: status, navio em operação, previsão de liberação, interrupção por clima
/fila Fila de Navios	clientes aguardando, ordenados por prioridade/IDA
/modelo Modelo de Fila	λ , μ , ρ , métricas M/M/1 por tipo e M/M/c (Erlang-C)
/simulacao Simulação	comparação das 4 políticas por DES (Wq, Lq, ρ , custo ponderado)
/recomendacao Recomendação	próxima atracação ótima + justificativa + timeline

Etapa 2 — Arquitetura técnica

api/_lib/	
baseDados.ts	tipos + nomes/tipos/capacidades dos berços
gerarDados.ts	SIMULAÇÃO M/G/4 ao vivo (snapshot estocástico no tempo)
clima.ts	clima real (Open-Meteo) + interrupção de operação
obterDados.ts	compõe simulação + clima (ponto único)
dados.ts	endpoint serverless (Vercel)
server/	coletor AIS (WebSocket aisstream) — navios reais opcionais
src/app/	
lib/filas/	
modelo.ts	parâmetros λ , μ , ρ + fórmulas M/M/1 e M/M/c (Erlang-C)
des.ts	Simulação de Eventos Discretos: compara 4 políticas
data/	

```
dataSource.ts      consume /api/dados
DadosContext.tsx   estado global + polling + botão Acelerar
pages/             as 6 telas
```

- **Estado:** React Context (DadosContext) com um único polling alimentando todas as telas; semente em mockData.ts como fallback.
- **Lógica de filas:** api/_lib/gerarDados.ts (sistema ao vivo) e src/app/lib/filas/ (modelo analítico + DES de comparação).
- **Bibliotecas:** React 18 + React Router 7, Vite 6, Tailwind v4, Recharts, lucide-react, motion; ws/tsx/dotenv (coletor AIS).

Etapa 3 — Teoria das Filas

Conceito	No sistema	Onde no código
Clientes	navios aguardando	fila em gerarDados / FilaNavios
Servidores (c=4)	os 4 berços	baseBercos, snapshot()
Chegada	Poisson por tipo (λ)	gerarChegadas() — intervalos exponenciais
Atendimento	exponencial por tipo (μ)	SERVICO_H, expo()
Saída	navio desatraca (berço→livre)	evento de fim de serviço
Disciplina	prioridade não preemptiva + compatibilidade	despachar() / selecionar()
Filas paralelas/restrrição	berço por tipo; B4 flexível (transbordo)	COMPAT
Capacidade	fila ilimitada (fundeadoiro)	sem bloqueio K

λ , μ , ρ são explícitos na aba Modelo de Fila (modelo.ts): por tipo (M/M/1) e agregado (M/M/c, Erlang-C: P_0 , $P(\text{espera})$, W_q , L_q , L , W). **Modelo:** M/G/4 com prioridade não preemptiva e servidores restritos. FIFO aparece apenas como baseline.

Etapa 4 — Processos Estocásticos

- **Chegadas aleatórias:** processo de Poisson (intervalos exponenciais) no simulador ao vivo e no DES; ou AIS real (aisstream) quando configurado.
- **Tempos de atendimento variáveis:** exponenciais com μ por tipo (Grãos 18h, Contêineres 12h, Combustíveis 10h, Multiuso 8h).
- **Clima:** entrada exógena estocástica real (Open-Meteo); quando Desfavorável, interrompe a operação dos berços ocupados (aplicarClima).
- **Evolução:** processo de nascimento-e-morte simulado por eventos discretos; o dashboard mostra um snapshot a cada polling (botão Acelerar adianta o relógio).

Honestidade: a simulação é semeada (reprodutível) — determinística dada a semente, mas o mecanismo é estocástico (Poisson + exponencial). O AIS, quando ligado, traz aleatoriedade do mundo real.

Etapa 5 — Requisitos de projeto (Parte 2)

#	Requisito	Evidência	Nota
1	Definir o modelo de fila	aba Modelo; gerarDados / des.ts	9
2	Identificar λ , μ , disciplina, capacidade, prioridades	modelo.ts; selecionar()	9
3	Justificar o tipo de fila	aba Simulação; des.ts	8.5
4	Simular cenários, prever, comparar políticas	des.ts; Simulacao.tsx	9
5	Propor política ótima, redução de congestionamento, redistribuição, plano crítico, recomendações	gerarDados; Recomendacao.tsx	8.5

Resultado-chave (defensável): entre políticas conservativas a espera média (W_q) é parecida (conservação de trabalho); o Índice Dinâmico vence onde importa — minimiza a espera dos navios críticos e o custo ponderado por prioridade, e o FIFO (fila única com bloqueio de cabeça) é claramente o pior.

Etapa 6 — Guia para apresentação

Simples: "Modelamos a fila de atracação do Itaqui e construímos o painel que a opera: ele simula navios chegando e atracando em 4 berços, prioriza por um índice e mostra que priorizar bem protege os navios críticos sem piorar o tempo médio."

Técnica: "M/G/4 com prioridade não preemptiva e restrição de berço. O backend é uma simulação de eventos discretos cujo snapshot alimenta o dashboard; a aba Simulação roda a mesma carga sob 4 políticas e mede W_q , L_q , ρ e custo ponderado."

Matemática: " λ por tipo (Poisson), μ por tipo (exponencial), $\rho = \lambda/\mu$. Na aba Modelo mostramos M/M/1 por berço e M/M/c agregado (Erlang-C: $P(\text{espera})$, W_q , L_q). O IDA é um score multicritério (0–100): prioridade + espera + tipo de carga + clima + impacto logístico + compatibilidade."

Perguntas prováveis e respostas:

- "O IDA reduz a espera média?" → "Não necessariamente — por conservação de trabalho, políticas conservativas empatam em W_q . O IDA reduz a espera ponderada e protege os críticos; é esse o objetivo da priorização."
- "Onde estão λ e μ ?" → "Na aba Modelo de Fila, por tipo e agregados."
- "O dashboard é real ou simulado?" → "Simulação estocástica M/G/4 ao vivo; opcionalmente, navios reais via AIS."
- "Como o clima entra?" → "Dado real do Open-Meteo; quando desfavorável, interrompe a operação dos berços."

Pontos fortes: modelo de filas implementado de verdade (DES + analítico); prioridade e restrição de berço reais; comparação honesta e reprodutível; dados reais opcionais (AIS, clima); arquitetura limpa e publicável.

Críticas possíveis e defesa:

- "Serviço exponencial é simplificação." → "Sim; a estrutura M/G permite trocar por log-normal/empírica sem mudar o motor."
- "Simulação semeada." → "Reprodutibilidade para a banca; o botão Re-simular troca a semente e

o AIS traz aleatoriedade real."

Etapa 7 — Resumo executivo

Desenvolvido: painel web (React/Vite/Tailwind) com backend de simulação de filas próprio. Dashboard ao vivo (M/G/4 estocástico), aba Modelo de Fila (λ , μ , ρ , M/M/1, Erlang-C), aba Simulação (DES comparando 4 políticas), Recomendação (IDA), navios reais via AIS e clima real (Open-Meteo).

Teoria das Filas: multi-servidor $c=4$; prioridade não preemptiva; filas paralelas com restrição/transbordo; comparação de políticas; métricas W_q , L_q , L , W , ρ , $P(\text{espera})$ (analíticas e simuladas).

Processos Estocásticos: chegadas de Poisson; serviço exponencial por tipo; clima exógeno real; processo de nascimento-e-morte via eventos discretos.

Requisitos: R1 9 · R2 9 · R3 8.5 · R4 9 · R5 8.5.

Melhorias futuras: tempos de serviço gerais calibrados com dados reais; prioridade contratual explícita por empresa/berço; persistência histórica para validar o modelo contra a operação real do porto.